

Q1

1. $T(x, y) = 5x^2 + 3y^2 + 4xy$

bound by line $2x + y \leq 12 = 0$
 $x \geq 0, y \geq 0$
 $y \leq 5$

2. $L(x, y, \lambda) = 5x^2 + 3y^2 + 4xy - \lambda(2x + y - 12)$

$\frac{\partial L}{\partial x} = 10x + 4y - 2\lambda = 0$
 $\boxed{\lambda = 5x + 2y} \Rightarrow A$

$\frac{\partial L}{\partial y} = 6y + 4x - \lambda = 0, \boxed{\lambda = 6y + 4x} \Rightarrow B$

$\frac{\partial L}{\partial \lambda} = 2x + y - 12 = 0, \boxed{y = 12 - 2x} \Rightarrow C$

Since $A=B$, $\therefore 5x + 2y = 6y + 4x, \boxed{4y = x} \Rightarrow D$

Substitute C into D: $4(12 - 2x) = x$
 $48 - 8x = x, \boxed{x = 5\frac{1}{3}}$
 $y = \frac{5\frac{1}{3}}{4}, \boxed{y = 1\frac{1}{3}}$

Since satisfying constraints, $\therefore \boxed{x = 16/3}$
 $\boxed{y = 4/3}$

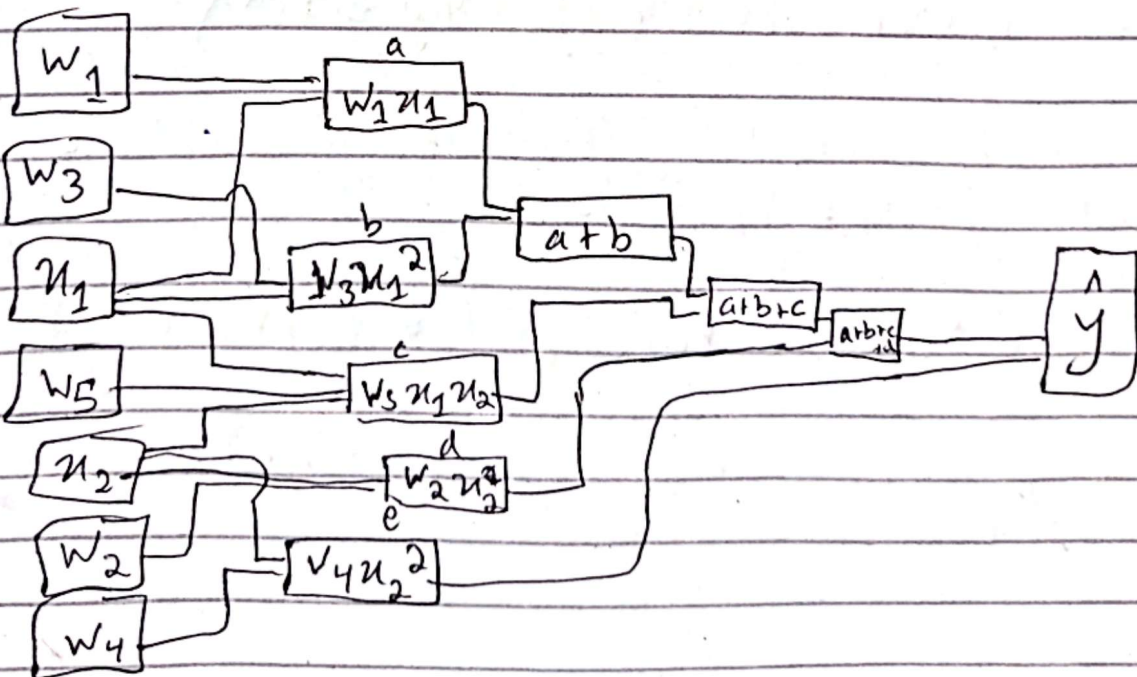
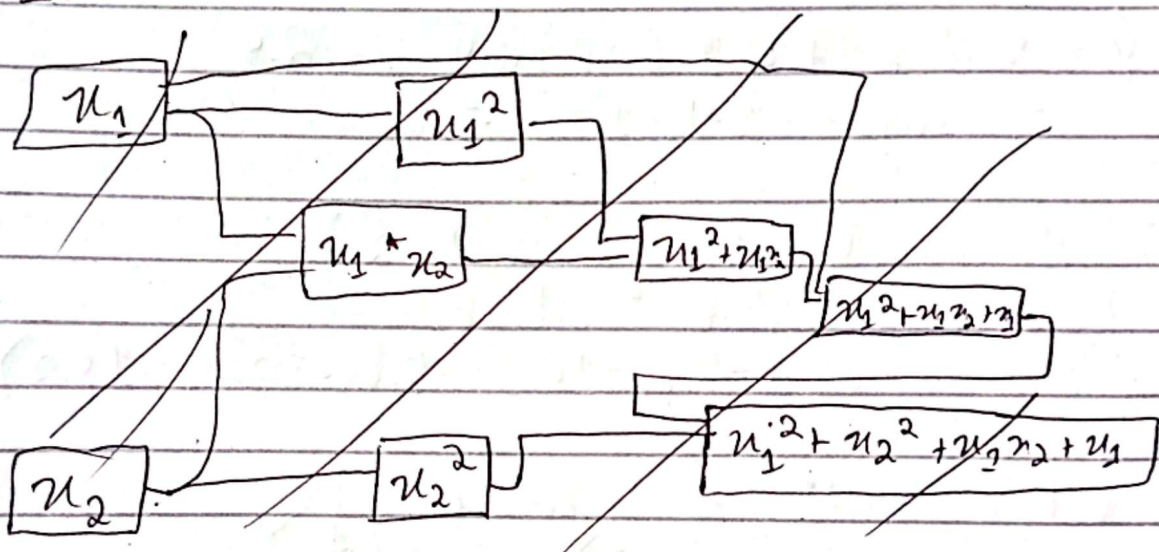
3. KKT: $-x \leq 0, -y \leq 0, y - 5 \leq 0$

Since all constraints are active, therefore no inequality constraints are active.

4. $f_x = 10x + 4y$ $f_y = 6y + 4x$
 $f_{xy} = 4 = f_{yx}$, $f_{xx} = 10$, $f_{yy} = 6$

$\therefore H = \begin{bmatrix} 10 & 4 \\ 4 & 6 \end{bmatrix}$, $|H| = 60 - 16$
 $= 44$, > 0 , $\therefore$ ^{care} ~~convex~~
 positive, $\therefore$ ~~convex~~

Q2
1.



$$2. \frac{\partial L}{\partial w_1} = n_1, \frac{\partial L}{\partial w_2} = n_2, \frac{\partial L}{\partial w_3} = 2n_1$$

$$\frac{\partial L}{\partial w_4} = 2n_2, \frac{\partial L}{\partial w_5} = n_1 n_2$$

$$\therefore \nabla_w L(w) = (n_1, n_2, 2n_1, 2n_2, n_1 n_2)$$

3. $n_1 = 1, n_2 = 2, y = 10$, weights given in q

$$\hat{y} = 0.5 + 1 + 0.1 + 0.4 + 0.4 = 2.4$$

$$\therefore \text{error} = 2.4 - 10 = -7.6$$

$\therefore$ if original = $(1, 2, \frac{1}{2}, 4, 2)$

then new = -7.6 * above

$$= (-7.6, -15.2, -7.6, -30.4, -15.2)$$

let $\lambda = 0.01$, $\therefore w_{\text{new}} =$

$$w_{\text{new}} = (0.5, 0.5, 0.1, 0.1, 0.2) - 0.01 \cdot (-7.6, -15.2, -7.6, -30.4, -15.2)$$

$$= (0.576, 0.652, 0.176, 0.404, 0.352)$$

4. Lipschitz condition is that there is a constant L such that $0 \leq \lambda \leq \frac{2}{L}$ and

$$\|\nabla_w L(w_a) - \nabla_w L(w_b)\| \leq L \|w_a - w_b\|$$

Q1 comparison of ~~gradient~~ Lagrange and KKT:
both give same outputs.

$$Q3) C(x) = ax^2 + Be^{-kx}$$

$$1. \quad C'(x) = 2ax - kBe^{-kx}$$

$$C''(x) = 2a - k^2 Be^{-kx}, \quad a, B, k \text{ constants}$$

$$= n - m \sqrt{e^{-x}}$$

$\therefore e^{-x} > 0$ for all x , $\therefore$ strictly convex.

$$2. \quad C'(x) = 2ax - kBe^{-kx}$$

$$= 2x - 2.5e^{-0.5}$$

$$= 2x - \frac{2.5}{\sqrt{e}}$$

$$3. \quad \text{Formula: } u_{n+1} = u_n - \lambda C'(u_n)$$

$$u_0 = 1, \text{ let } \lambda = 0.1$$

$$\therefore u_{n+1} = 1 - 0.1 \left(2 - \frac{2.5}{e} \right)$$

$$= 1 - 0.0484$$

$$= 0.9516$$

4. λ must be large enough to reach solution in efficient steps, yet small enough to not overshoot. This is avoided by changing the value of λ every iteration.

5. First, start with a large value of λ and decrease it with reference to the Armijo condition until it is sufficient.

Q4) $f(x, y) = e^{-0.1x} \cos(0.5y) + 2xy^2$

$f(x, y) = f(0, 0) + f$

$f(0, 0) = 1 \times 1 + 0 = 1$

$f_x = -0.1 e^{-0.1x} + y^2$

$f_x(0, 0) = -0.1 + 0 = -0.1$

$f_y = \sin(0.5y) + 2xy$

$f_y(0, 0) = 0$

$f_{xx} = 0.01 e^{-0.1x} = 0.01$

$f_{yy} = -0.25 \cos(0.5y) + 2x = -0.25$

$f_{xy} = f_{yx} = 2y = 0$

$\therefore f(x, y) = 1 + (-0.1)x + 0y + \frac{1}{2} (0.01x^2 + 2(-0.25)xy + 0.01y^2)$
 $= 1 - 0.1x - 0.125xy^2 + 0.005x^2$
 $= \boxed{1 + 0.005x^2 - 0.1x - 0.125y^2}$

Linear Effect due to $-0.1x$, Hence same processing delay linearly slows performance

Quadratic Effect minimally from x and more from y ,
 therefore temperature is more damaging.

Q1

```
No Python interpreters found | A3MVC.py* [icon] [X]
1  import numpy as np
2
3  def objective(x, y):
4      return 5 * x**2 + 3 * y**2 + 4 * x * y
5
6  def solve_numerical():
7
8      x_values = np.linspace(3.5, 6, 1000)
9      best_T = -np.inf
10     best_x = -1
11     best_y = -1
12
13     for x in x_values:
14         y = 12 - 2 * x
15         T = objective(x, y)
16         if T > best_T:
17             best_T = T
18             best_x = x
19             best_y = y
20
21     print(f"Numerical Solution:")
22     print(f"Optimal x: {best_x}")
23     print(f"Optimal y: {best_y}")
24     print(f"Max Throughput T: {best_T}")
25
26     solve_numerical()
```

```
Python 3.10.0 (tags/v3.10.0:2016.12.22) [AMD64]
C:\Windows\system32\cmd.exe
Numerical Solution:
Optimal x: 3.5
Optimal y: 5.0
Max Throughput T: 206.25
Press any key to continue . . .
```

Q2

No Python interpreters found

A3MVC.py

```
1 from sympy import symbols, diff
2
3 x1, x2, w1, w2, w3, w4, w5 = symbols('x1 x2 w1 w2 w3 w4 w5')
4 y_hat = w1*x1 + w2*x2 + w3*x1**2 + w4*x2**2 + w5*x1*x2
5
6 print(f"Symbolic y_hat: {y_hat}")
7 values = {
8     x1: 1,
9     x2: 2,
10    w1: 0.5,
11    w2: 0.5,
12    w3: 0.1,
13    w4: 0.1,
14    w5: 0.2
15 }
16 y_hat_value = y_hat.subs(values)
17 print(f"Numerical y_hat: {y_hat_value:.2f}")
18
19 y = symbols('y')
20
21 loss = (1/2) * (y_hat - y)**2
22 print(f"Symbolic Loss: {loss}")
23
24 gradient_L = [
25     diff(loss, w1),
26     diff(loss, w2),
27     diff(loss, w3),
28     diff(loss, w4),
29     diff(loss, w5)
30 ]
31 print(f"Symbolic Gradient L/w1: {gradient_L[0]}")
32 print(f"Symbolic Gradient L/w2: {gradient_L[1]}")
33 print(f"Symbolic Gradient L/w3: {gradient_L[2]}")
34 print(f"Symbolic Gradient L/w4: {gradient_L[3]}")
35 print(f"Symbolic Gradient L/w5: {gradient_L[4]}")
36 values[y] = 10
37
38 gradient_values = [grad.subs(values) for grad in gradient_L]
39 print(f"Numerical Gradient Vector: {gradient_values}")
40
41 learning_rate = 0.01
42
43 current_weights = [values[w1], values[w2], values[w3], values[w4], values[w5]]
44 print(f"Current Weights: {current_weights}")
45
46 new_weights = [cw - learning_rate * gv for cw, gv in zip(current_weights, gradient_values)]
47 print(f"New Weights: {new_weights}")
```

C:\Windows\system32\cmd.exe

```
Symbolic y_hat: w1*x1 + w2*x2 + w3*x1**2 + w4*x2**2 + w5*x1*x2
Numerical y_hat: 2.40
Symbolic Loss: 0.5*(w1*x1 + w2*x2 + w3*x1**2 + w4*x2**2 + w5*x1*x2 - y)**2
Symbolic Gradient L/w1: 1.0*x1*(w1*x1 + w2*x2 + w3*x1**2 + w4*x2**2 + w5*x1*x2 - y)
Symbolic Gradient L/w2: 1.0*x2*(w1*x1 + w2*x2 + w3*x1**2 + w4*x2**2 + w5*x1*x2 - y)
Symbolic Gradient L/w3: 1.0*x1**2*(w1*x1 + w2*x2 + w3*x1**2 + w4*x2**2 + w5*x1*x2 - y)
Symbolic Gradient L/w4: 1.0*x2**2*(w1*x1 + w2*x2 + w3*x1**2 + w4*x2**2 + w5*x1*x2 - y)
Symbolic Gradient L/w5: 1.0*x1*x2*(w1*x1 + w2*x2 + w3*x1**2 + w4*x2**2 + w5*x1*x2 - y)
Numerical Gradient Vector: [-7.600000000000000, -15.200000000000000, -7.600000000000000, -30.400000000000000, -15.200000000000000]
Current Weights: [0.5, 0.5, 0.1, 0.1, 0.2]
New Weights: [0.5760000000000000, 0.6520000000000000, 0.1760000000000000, 0.4040000000000000, 0.3520000000000000]
Press any key to continue . . .
```